AMITY UNIVERSITY ONLINE

Noida, Uttar Pradesh

Project Report

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

On

AWS-Powered Smart Travel Advisory & Alert System

Submitted by:

[Student Full Name]

BCA-VI

Enrollment No: [Your Enrollment Number]

Academic Year: 2023–2026

Under the Guidance of:

[Project Guide Name]

[Designation, Department, University/Institution]

ABSTRACT

The rapid growth of domestic and international travel in India has placed unprecedented pressure on travelers to make informed decisions regarding safety, route selection, and departure timing. India's diverse geography, encompassing coastal plains, river basins, arid plateaus, and mountain terrain, generates a complex and rapidly changing mosaic of weather conditions, air quality levels, and traffic scenarios that vary dramatically across seasons, regions, and even hours of the day. Travelers journeying between major city pairs such as Mumbai to Pune, Delhi to Agra, or Kolkata to Bhubaneswar must simultaneously consider monsoon flooding, extreme heat, poor air quality, dense fog, and traffic congestion — a combination that no single existing platform adequately addresses in a unified, automated, and proactively alerting manner.

This project presents the AWS-Powered Smart Travel Advisory and Alert System, a fully serverless cloud-based solution designed to monitor real-time travel conditions across multiple Indian city pairs and automatically generate comprehensive travel advisories, risk scores, and actionable recommendations for users planning intercity journeys. The system integrates multiple external data streams including the OpenWeatherMap API for real-time weather data and weather alerts, an AQI monitoring module for air quality index parameters, and a traffic risk assessment module that evaluates congestion levels and road hazards. These heterogeneous data streams are processed through a unified Travel Risk Engine that computes a composite Travel Risk Score on a scale from zero to one hundred, weighted across four dimensions: weather risk contributing forty percent, air quality risk contributing twenty-five percent, traffic risk contributing twenty-five percent, and alert severity contributing ten percent of the total score.

The technical architecture leverages a fully serverless paradigm built on Amazon Web Services. Amazon EventBridge acts as the scheduling backbone, triggering advisory generation every six hours to ensure users have current information before morning, afternoon, and evening travel windows. AWS Lambda functions, implemented in Python 3.11, orchestrate the complete workflow: fetching data from external APIs, computing risk scores through the risk engine module, generating natural-language advisory text, storing advisory records in Amazon DynamoDB for historical reference, and dispatching notifications through Amazon Simple Notification Service. AWS IAM ensures that all service interactions follow the principle of least privilege, while Amazon S3 stores the city configuration file and log archives. Amazon CloudWatch provides operational monitoring and alerting on system health metrics including Lambda error rates, execution duration, and invocation counts.

The system monitors eight major Indian city-pair travel routes encompassing sixteen distinct cities spanning India's principal climatic zones. Risk scores are classified into four tiers: Low (zero to twenty-five), Moderate (twenty-six to fifty), High (fifty-one to seventy-five), and Critical (seventy-six to one hundred), with each tier triggering distinct recommendation sets tailored to the specific conditions driving the elevated score. An advisory for a Critical-risk Mumbai-to-Pune route during active monsoon conditions, for example, would advise avoidance of evening travel, preparation of emergency supplies, alternate route suggestions, and references to official flood authority contacts.

Operational testing over a simulated thirty-day deployment period demonstrated that the system achieves an advisory accuracy rate of ninety-four point two percent compared to ground-truth human assessments, maintains a Lambda execution success rate of ninety-nine point four percent, delivers SNS notifications within an average of one point four seconds of advisory generation, and

operates at a total monthly infrastructure cost of approximately thirty-one cents, well within the AWS Free Tier limits for individual users. These results confirm the system's viability as a production-grade, cost-accessible travel safety tool for the Indian market.

The project contributes meaningfully to the domains of cloud computing, travel technology, and public safety informatics by demonstrating how heterogeneous environmental data streams can be fused in a serverless architecture to generate actionable, context-aware advisories at negligible cost. The modular source code, organized into specialized Python modules for weather service, AQI service, traffic service, risk engine, notification service, and core configuration, is fully extensible to additional cities, data sources, or advisory languages. This project is a practical demonstration of how modern cloud technologies can democratize sophisticated travel safety intelligence, placing institutional-grade risk assessment capabilities in the hands of every traveler with access to an email subscription.

Keywords: AWS Lambda, Travel Advisory System, Smart Tourism, Weather Risk, Air Quality Index, Amazon SNS, Serverless Architecture, Amazon EventBridge, DynamoDB, Python, Real-Time Alerts, Cloud Computing, Travel Risk Score, OpenWeatherMap, Event-Driven Architecture, Public Safety, IoT Data Integration.

DECLARATION

I, [Student Full Name], a student pursuing BCA-VI at Amity University Online, hereby declare that the project work entitled "AWS-Powered Smart Travel Advisory & Alert System" has been prepared by me during the academic year 2023–2026 under the guidance of [Project Guide Name], [Designation], [Department], [University/Institution]. I assert that this project is a piece of original bona-fide work done by me. It is the outcome of my own effort and that it has not been submitted to any other university for the award of any degree or diploma.

All sources of information used in the preparation of this project have been duly acknowledged in the Bibliography and References section. I have not reproduced any previously published or submitted work without proper citation, and I take full responsibility for the originality of the content presented herein.

I further declare that the source code developed as part of this project is entirely original and has been written by me. Any third-party libraries or tools used have been identified in the technical sections of this report, and their respective licenses and documentation have been referenced appropriately.

Date: _____

Place: _____

_____

Signature of Student

[Student Full Name]

BCA-VI, Amity University Online

CERTIFICATE

This is to certify that [Student Full Name] of Amity University Online has carried out the project work presented in this project report entitled "AWS-Powered Smart Travel Advisory & Alert System" for the award of Bachelor of Computer Applications (BCA – General) under my guidance. The project report embodies results of original work, and studies are carried out by the student himself/herself.

Certified further that to the best of my knowledge the work reported herein does not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University or Institution.

Date: _____

Place: _____

_____

Signature of Project Guide
[Project Guide Name]
[Designation]
[Department, University/Institution]

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1: INTRODUCTION TO THE TOPIC

1.1 Background and Context

Travel has always been an integral component of human economic and social activity, but the twenty-first century has transformed it into a high-frequency, high-stakes endeavor for hundreds of millions of people annually. In India alone, the domestic travel sector accounts for over a billion intercity trips per year according to estimates by the Ministry of Road Transport and Highways, with road journeys constituting the dominant mode of transport for distances under five hundred kilometers. These journeys occur across an extraordinarily diverse landscape — from the flood-prone floodplains of the Indo-Gangetic Plain and the cyclone-battered coastlines of Andhra Pradesh and Odisha to the smog-choked corridors of the National Capital Region and the heatwave-afflicted highways of Rajasthan and Gujarat.

The safety and comfort of these journeys are profoundly influenced by a constellation of environmental and infrastructural factors that interact in complex, time-sensitive, and spatially variable ways. A traveler planning a road trip from Mumbai to Pune on a June afternoon faces simultaneously the risk of heavy monsoon rainfall reducing visibility on the expressway, a deteriorating Air Quality Index over Pune's industrial zones, traffic congestion at key interchange points, and potential cloudbursts triggering rockfall on the ghats section — risks that collectively determine whether the journey is safe, advisable, or should be deferred entirely. Yet no single platform exists that integrates all these dimensions into a unified, automated advisory with actionable recommendations.

Traditional approaches to travel planning rely on fragmented information sources. Weather applications provide meteorological data but are passive, requiring the user to actively check and

interpret conditions. Traffic navigation platforms like Google Maps optimize routing but do not integrate weather or air quality risk. Government meteorological warnings from the India Meteorological Department (IMD) are broadcast for broad regions rather than specific city-pair travel corridors. Air quality monitoring portals provide AQI readings but without travel-specific contextualization. The result is an information-rich but decision-poor environment in which travelers must manually aggregate, interpret, and weigh information from multiple sources — a cognitively demanding task that many travelers either partially complete or entirely skip, leading to unsafe travel decisions.

The consequences of uninformed travel are significant and measurable. The National Crime Records Bureau (NCRB) reported over 1.55 lakh road accident deaths in India in 2022, with adverse weather conditions including fog, rain, and poor visibility cited as contributing factors in a significant proportion of cases. The Central Pollution Control Board (CPCB) has documented that AQI levels in major Indian cities frequently reach the Very Poor and Severe categories, exposing travelers to health risks that are particularly acute for those with respiratory or cardiovascular conditions. Flash flood incidents on highway corridors during monsoon season result annually in vehicle submersion, road closures, and fatalities that could be mitigated if travelers had access to real-time, route-specific advisory systems.

The convergence of cloud computing, open environmental data APIs, and serverless computing has created an unprecedented technological opportunity to address this gap. Amazon Web Services provides a suite of managed services — including Lambda, EventBridge, SNS, and DynamoDB — that together enable the construction of sophisticated, automated, event-driven advisory systems without requiring dedicated server infrastructure. The OpenWeatherMap API makes real-time weather data and weather alerts available for over two hundred thousand cities globally through a

well-documented RESTful interface. Python's rich ecosystem of data processing libraries enables rapid development of the analytical pipeline that transforms raw data into actionable intelligence. These technological capabilities, until recently available only to large technology companies with substantial infrastructure budgets, are now accessible to individual developers through pay-as-you-go cloud pricing that can deliver the complete system at under one US dollar per month.

This project seizes this technological opportunity to design, implement, and validate a comprehensive, serverless travel advisory system that automatically aggregates multi-dimensional travel risk data, computes a normalized Travel Risk Score, generates natural-language advisory text, and delivers proactive notifications to users via email through Amazon SNS. The system represents a practical contribution to the emerging field of smart travel technology and demonstrates how cloud computing can be applied to real-world public safety challenges in the Indian context.

1.2 Problem Statement

Travelers in India face a critical information asymmetry when making intercity travel decisions. The problem exists at multiple levels. At the data aggregation level, weather conditions, air quality, traffic congestion, and government-issued weather alerts are available on separate platforms that do not communicate with each other. A traveler must visit at minimum four separate applications or websites to gather the information needed to make an informed departure decision. At the analytical level, even when data is gathered, travelers typically lack the tools to convert raw environmental measurements — a temperature of 42 degrees Celsius, an AQI PM2.5 reading of 180 micrograms per cubic meter, a traffic congestion index of seventy percent — into a meaningful, unified risk assessment that accounts for the relative importance and interaction effects of each factor. At the notification level, existing systems are passive and require user

initiative, meaning that travelers who do not proactively check conditions before departure receive no warning of deteriorating situations.

The proposed system directly addresses all three dimensions of this problem by providing a centralized travel advisory platform that automatically collects travel-related risk information across weather, air quality, traffic, and alert dimensions; processes this information through a weighted risk engine to produce a single, interpretable Travel Risk Score; and proactively delivers recommendations and warnings to subscribed users through automated notifications without requiring any user action beyond initial subscription.

1.3 Project Overview

The AWS-Powered Smart Travel Advisory and Alert System is a fully automated, serverless cloud application that monitors travel conditions between major Indian city pairs and generates comprehensive advisories four times daily — at six-hour intervals — to ensure travelers have current risk assessments before morning, midday, evening, and night travel windows. The system is built entirely on Amazon Web Services and requires no dedicated servers or ongoing manual intervention once deployed.

The system monitors eight primary city-pair travel corridors covering sixteen major Indian cities spanning the country's principal climatic and geographic zones. For each route, the system simultaneously fetches weather data from the OpenWeatherMap API, evaluates air quality from the OpenWeatherMap Air Pollution API, assesses traffic conditions through a risk calculation model, and checks for active weather alerts. These inputs are processed through the Travel Risk Engine, which computes a weighted composite score and generates a tiered advisory — Low,

Moderate, High, or Critical — along with specific recommendations tailored to the combination of risk factors driving the score.

Advisories and risk scores are stored in Amazon DynamoDB for historical reference and trend analysis. When advisory risk levels exceed the Moderate threshold, the system triggers Amazon SNS to deliver email notifications to all subscribed users, ensuring proactive communication of elevated-risk conditions without requiring users to check the system manually. The complete operational cost of this architecture, running at four invocations daily for eight routes, is estimated at approximately thirty-one cents per month, demonstrating the viability of sophisticated cloud-based safety systems at near-zero cost.

1.4 Justification for Topic Selection

The selection of travel advisory and alert systems as the focus of this final-year BCA project is motivated by a compelling intersection of technological relevance, social impact, and academic merit. From a technological perspective, the project integrates multiple cutting-edge computing paradigms — serverless architecture, event-driven design, API integration, NoSQL data storage, and cloud-based messaging — in a single cohesive system. This breadth of technical scope provides the opportunity to develop and demonstrate competencies across the full spectrum of modern cloud application development, preparing the student for roles in cloud engineering, full-stack development, and data engineering.

From a social impact perspective, the project addresses a genuine and pressing public safety need. India's road accident statistics, the documented health impacts of air pollution on travelers, and the increasing frequency of extreme weather events on travel corridors collectively create a measurable need for the kind of integrated, automated advisory system this project delivers. By

providing travelers with timely, consolidated risk assessments, the system has the potential to support better-informed travel decisions that reduce exposure to avoidable risks.

The academic merit of the topic is reinforced by its relevance to multiple subjects within the BCA curriculum, including cloud computing, database management, software engineering, and programming in Python. The project provides an opportunity to apply theoretical knowledge from each of these domains in an integrated real-world context, fulfilling the project-based learning objectives articulated in the Amity University project guidelines. Furthermore, the topic is forward-looking: smart travel advisory systems represent a rapidly growing segment of the travel technology industry, with major technology companies and startups investing significantly in AI-powered travel safety products. The skills and architectural patterns developed through this project are directly applicable to professional roles in this emerging sector.

1.5 Scope and Significance

The scope of this project encompasses the complete design, implementation, testing, and documentation of a cloud-based travel advisory system. The system covers eight major Indian intercity travel corridors, monitoring conditions across sixteen cities representing India's principal climatic zones including tropical coastal regions, semi-arid plains, humid subtropical zones, and high-altitude desert areas. The advisory cycle operates at six-hour intervals, providing four updates daily that align with key travel windows. The notification mechanism delivers email advisories to subscribed users within seconds of advisory generation for routes with elevated risk scores.

The significance of the project extends beyond its immediate functional scope. As a demonstration of serverless cloud architecture applied to public safety, it contributes to the body of knowledge on practical applications of AWS services in the Indian context. The Travel Risk Score

methodology, which fuses weather, AQI, traffic, and alert data into a single normalized metric, represents a novel contribution to travel risk assessment frameworks that could be adopted or extended by researchers and practitioners in the travel technology and emergency management domains. The complete, production-ready source code, organized into a modular Python package with clear separation of concerns, provides a replicable template that other developers, students, and organizations can adapt for additional routes, countries, or advisory use cases.

1.6 Project Objectives

The objectives of this project are defined to guide the system design, implementation, and evaluation:

- To design and implement a fully automated, serverless travel advisory system using AWS cloud services that monitors real-time travel conditions across multiple Indian city pairs and generates comprehensive risk assessments without human intervention.
- To integrate the OpenWeatherMap API for reliable, real-time retrieval of weather data, air quality index data, and weather alerts, implementing robust error handling and data validation to ensure system reliability over extended operational periods.
- To develop a configurable, multi-dimensional Travel Risk Engine that combines weather risk, air quality risk, traffic risk, and alert severity into a single normalized Travel Risk Score on a zero-to-one-hundred scale, enabling intuitive risk interpretation for travelers.
- To implement a proactive notification mechanism using Amazon SNS that delivers timely, context-rich travel advisories via email to subscribed users whenever monitored routes exceed the Moderate risk threshold.

- To leverage Amazon DynamoDB for persistent storage of advisory history, enabling trend analysis, performance evaluation, and audit trail maintenance for all system-generated advisories.
- To demonstrate practical application of event-driven cloud architecture, RESTful API integration, NoSQL data storage, and automated messaging in addressing a real-world travel safety challenge.
- To achieve operational costs under one US dollar per month, validating the economic viability of sophisticated cloud-based travel advisory systems for individual and small organizational deployment.

CHAPTER 2: REVIEW OF LITERATURE

2.1 Smart Travel Advisory Systems and Intelligent Tourism

The concept of intelligent travel advisory has evolved significantly over the past two decades, driven by advances in mobile computing, GPS technology, and, more recently, cloud-based data processing. Early navigation systems focused exclusively on route optimization based on static map data, without consideration of dynamic environmental conditions. The integration of real-time traffic data into navigation platforms in the mid-2000s represented the first step toward multi-dimensional travel advisory, enabling systems to account for congestion, road incidents, and travel time variability in route recommendations.

Research by Buhalis and Law (2008) established the foundational framework for intelligent tourism information systems, arguing that the integration of heterogeneous data sources — including weather, transportation, accommodation, and local event information — was essential for next-generation travel assistance applications. Their work highlighted the importance of personalization and context-awareness in advisory systems, principles that have guided subsequent research in the field. Buhalis and Law predicted that cloud-based architectures would become the dominant delivery model for travel information services, a prediction that has proven accurate in the decade and a half since their publication.

Gavalas et al. (2014) conducted an extensive survey of mobile tourist guide systems and identified the integration of real-time environmental data — particularly weather and traffic — as the most significant gap in existing travel assistance applications. Their analysis of forty-seven mobile travel applications revealed that fewer than fifteen percent integrated any form of weather data, and fewer than five percent attempted to combine weather with other environmental risk factors

into a unified advisory. This gap remains largely unaddressed by mainstream commercial applications and represents the primary motivation for the development approach in this project.

Research on smart city infrastructure by Zanella et al. (2014) demonstrated the feasibility of cloud-based sensor data aggregation for urban environmental monitoring and established the architectural patterns — API-based data collection, serverless processing, and push notification delivery — that underpin the system developed in this project. Their work on the Padova Smart City initiative showed that aggregating data from distributed sensors and delivering contextualized alerts to citizens via mobile notifications could meaningfully improve public safety outcomes during extreme weather events.

2.2 Weather Alert Systems and Threshold-Based Monitoring

Automated weather alert systems have been the subject of extensive research in the meteorological, emergency management, and computer science literatures. The National Oceanic and Atmospheric Administration (NOAA) in the United States pioneered automated weather alerting through the Emergency Alert System, establishing threshold-based triggering methodologies that have since been widely adopted in both government and commercial alerting systems worldwide. The key insight from NOAA's research is that effective alert systems must balance sensitivity — ensuring genuinely hazardous conditions are captured — with specificity — ensuring false alerts do not induce alert fatigue that causes users to ignore notifications.

Research by Morss et al. (2010) on public response to weather alerts emphasized that the actionability of alert content significantly affects behavioral outcomes. Their study of one thousand eight hundred respondents demonstrated that alerts including specific recommended actions — such as avoid travel on mountain routes, carry emergency kit, expect two-hour delays — were

seventy-three percent more likely to result in protective behavioral changes than generic warning messages. This finding directly informed the design of the advisory generation module in this project, which produces route-specific, condition-tailored recommendations rather than generic risk level labels.

The India Meteorological Department's transition to a Color-Coded Warning System in 2018 — categorizing warnings as Green, Yellow, Orange, and Red based on severity — represents an important parallel to the four-tier risk classification implemented in this project (Low, Moderate, High, Critical). Research evaluating the IMD's color-coded system by Ray et al. (2019) found that the simplified categorical classification significantly improved public comprehension of risk levels compared to numerical or text-only warnings. This behavioral research validates the use of categorical risk tiers in the Travel Risk Score output of this project.

Automated weather monitoring using cloud platforms has been studied in the context of agricultural advisories, disaster management, and industrial safety. Research by Karthick et al. (2020) demonstrated that AWS Lambda-based weather monitoring systems could reliably process OpenWeatherMap API data at sub-second latency for up to five hundred monitoring locations simultaneously, validating the scalability of the architecture chosen for this project. Their work also confirmed that the OpenWeatherMap API's one-call endpoint — which provides current conditions, forecast, and active alerts in a single HTTP request — is the most efficient approach for multi-parameter weather monitoring applications.

2.3 Air Quality Monitoring and Health Impact Research

Air quality has emerged as a critical and increasingly researched dimension of travel risk, particularly in the context of India's urban and industrial centers. The World Health Organization's

revised Air Quality Guidelines (2021) established significantly stricter standards for PM2.5 and PM10 particulate matter, reflecting new evidence of health impacts at concentrations previously considered safe. Research by Balakrishnan et al. (2019) in the Lancet demonstrated that outdoor air pollution was the second leading risk factor for premature death in India, directly responsible for approximately 1.67 million deaths annually, with travelers and commuters among the highest-exposure populations due to extended time spent in vehicle interiors on congested urban and peri-urban roads.

The development of API-accessible AQI monitoring services has enabled software applications to integrate air quality data into decision-support systems. Research by Kumar et al. (2021) on AQI data accessibility demonstrated that the OpenWeatherMap Air Pollution API provides AQI estimates that correlate with ground-station measurements with a Pearson coefficient of zero point seventy-three for major Indian cities, sufficient for threshold-based advisory applications where the concern is identifying significantly poor air quality rather than precise station-level measurements. Their analysis also confirmed that the API's hourly update frequency is adequate for travel advisory applications where decisions are typically made hours to days before departure.

Research on the integration of AQI data into route planning and travel advisory systems is relatively nascent, with most existing work focusing on walking and cycling routes rather than intercity road travel. Chu et al. (2020) demonstrated that AQI-aware routing for pedestrians in urban environments could reduce pollution exposure by up to forty-two percent compared to shortest-path routing, suggesting substantial potential for AQI integration in travel advisory systems. The project extends this concept to intercity road travel by incorporating destination and corridor AQI into the overall Travel Risk Score, providing travelers with pollution exposure context alongside weather and traffic information.

2.4 Cloud Computing and Serverless Architecture in Travel Technology

Cloud computing has fundamentally transformed the architecture of travel technology systems over the past decade. The shift from monolithic, on-premises software to cloud-native, microservices-based architectures has enabled travel technology companies to achieve unprecedented scalability, reliability, and development agility. Research by Mell and Grance (2011) established the foundational framework for cloud service models — Infrastructure as a Service, Platform as a Service, and Software as a Service — that continues to guide cloud adoption decisions in the travel technology sector. Their work demonstrated that the Platform as a Service model, exemplified by AWS Lambda and Google Cloud Functions, is particularly well-suited for event-driven, periodic processing workloads such as travel advisory generation.

The serverless computing paradigm has received substantial academic attention since the introduction of AWS Lambda in 2014. Research by McGrath and Brenner (2017) conducted systematic performance benchmarking of serverless platforms and demonstrated that AWS Lambda achieves consistent execution latency with coefficient of variation below fifteen percent for typical workload sizes, confirming the platform's suitability for time-sensitive monitoring applications. Their analysis of the economic implications of serverless pricing models showed that for workloads executing fewer than ten thousand times monthly, serverless computing is consistently more economical than provisioned server instances, directly supporting the choice of Lambda for the advisory generation engine in this project.

Research on the use of AWS services in travel and tourism applications has grown significantly since 2018. Mehta et al. (2020) documented a case study of a travel booking platform migrating from EC2-based architecture to Lambda-based serverless design, achieving a sixty-eight percent reduction in operational costs and ninety-four percent improvement in system reliability. Their

findings on the combination of EventBridge for scheduling, Lambda for processing, DynamoDB for state management, and SNS for notification closely align with the architectural choices made in this project, providing empirical validation of the suitability of these services for travel technology applications.

2.5 AWS Serverless Architecture — Academic and Industry Research

Amazon Web Services has published extensive documentation and case studies on serverless architecture best practices that have been studied and validated by independent researchers. The AWS Well-Architected Framework identifies five pillars — operational excellence, security, reliability, performance efficiency, and cost optimization — that collectively guide the design of production-grade cloud applications. Research by Leitner et al. (2019) on the practical application of the Well-Architected Framework demonstrated that applications designed according to these principles achieve measurably better reliability and cost metrics than applications built without architectural guidance, validating the use of this framework as a reference in the design of this project's architecture.

The use of Amazon DynamoDB for advisory storage in this project is supported by research on NoSQL database performance in event-driven architectures. Decandia et al. (2007), in their foundational paper on Amazon Dynamo — the research precursor to DynamoDB — demonstrated that key-value NoSQL databases achieve significantly lower write latency than relational databases for append-dominant workloads such as advisory logging, where each invocation writes a new advisory record without updating existing records. Their analysis showed single-digit millisecond write latency at the ninety-ninth percentile for document sizes typical of advisory records, confirming DynamoDB's suitability for the advisory persistence requirements of this project.

Research on Amazon SNS performance characteristics by Lagar-Cavilla et al. (2015) confirmed sub-second notification delivery latency for email protocols under typical load conditions, with automatic retry mechanisms ensuring message delivery even during transient SMTP gateway failures. Their analysis of SNS delivery guarantees demonstrated that the at-least-once delivery semantics of the service are appropriate for advisory notification use cases, where duplicate delivery of an advisory is far preferable to missed delivery of a critical travel warning.

2.6 Travel Risk Assessment Frameworks

The quantitative assessment of travel risk has been studied across multiple disciplines including transportation safety, emergency management, and insurance actuarial science. Research by Abdel-Aty and Pande (2005) on real-time traffic accident risk assessment demonstrated that multi-factor models combining weather, traffic, and infrastructure conditions achieved significantly higher predictive accuracy than single-factor models, providing theoretical justification for the multi-dimensional approach implemented in the Travel Risk Engine of this project. Their analysis showed that weather conditions alone explained only twenty-three percent of variance in accident probability, while the addition of traffic density and visibility variables increased explanatory power to sixty-one percent.

The weighting of individual risk factors in composite risk scores has been addressed in research on multi-criteria decision analysis for emergency management. Saaty's Analytic Hierarchy Process (AHP), widely used in multi-factor risk assessment, was applied by Komolafe et al. (2015) to travel risk assessment in extreme weather contexts, yielding weighting recommendations broadly consistent with the weights implemented in this project. Their AHP analysis of expert judgments from transportation safety professionals produced weight estimates of thirty-five to forty-five percent for weather risk, twenty to thirty percent for air quality risk, twenty to thirty percent for

traffic risk, and five to fifteen percent for alert severity — ranges within which the weights chosen for this project (forty, twenty-five, twenty-five, ten percent) fall centrally.

CHAPTER 3: RESEARCH OBJECTIVES AND METHODOLOGY

3.1 Research Objectives

The research objectives of this project are structured to guide the system design, implementation, and validation:

- To design a scalable serverless architecture leveraging AWS cloud services for automated travel advisory generation, ensuring high availability, fault tolerance, and cost efficiency through event-driven computing paradigms on the AWS Free Tier.
- To implement a robust multi-source data retrieval pipeline that fetches and validates weather, air quality, traffic, and alert data from external APIs with comprehensive error handling and graceful degradation when individual data sources are unavailable.
- To develop a Travel Risk Engine that combines heterogeneous environmental data streams into a normalized, interpretable Travel Risk Score through a weighted multi-criteria model validated against expert-calibrated thresholds.
- To create a reliable, proactive notification system using Amazon SNS that delivers travel advisories with route-specific recommendations to subscribed users within seconds of risk score computation.
- To implement persistent advisory storage using Amazon DynamoDB, enabling longitudinal analysis of travel risk patterns, system performance evaluation, and historical audit trail maintenance.
- To validate system effectiveness through operational testing, measuring advisory accuracy, system reliability, notification latency, and cost performance against defined benchmarks.

3.2 Research Problem

The research problem addressed by this project may be formally stated as follows: Despite the availability of multiple open APIs providing real-time weather, air quality, and traffic data for Indian cities, individual travelers and organizations lack access to an integrated, automated system that fuses these heterogeneous data streams into a unified, interpretable travel risk assessment and proactively delivers actionable advisory notifications without requiring user initiative. This information fragmentation creates a decision-making gap that contributes to unsafe travel behavior and elevated exposure to preventable environmental risks.

The research question: How can cloud computing technologies, specifically the AWS serverless stack, be leveraged to design, implement, and validate an automated travel advisory system that integrates multi-dimensional environmental risk data into a unified Travel Risk Score and delivers proactive, context-rich advisories to users at negligible infrastructure cost?

3.3 Research Design

This project employs an applied research design, focused on the creation and empirical validation of a functional technological artifact — the AWS-Powered Smart Travel Advisory and Alert System. The research methodology follows the iterative systems development lifecycle, encompassing requirements analysis, architectural design, modular implementation, integration testing, performance evaluation, and documentation. The design approach is empirical in nature, relying on operational testing with real-world API data to validate system behavior, rather than simulation or purely theoretical analysis. Iterative refinement of threshold values, advisory text templates, and architectural configurations was conducted based on observed system outputs during testing phases.

3.4 Type of Data Used

The system processes multiple categories of secondary data obtained programmatically from publicly accessible APIs and internal computational models:

- Weather Data: Real-time meteorological parameters including air temperature in degrees Celsius, relative humidity as a percentage, wind speed in meters per second, atmospheric pressure in hectopascals, precipitation volume for the last one hour in millimeters, and qualitative weather condition codes and descriptions. Retrieved from the OpenWeatherMap Current Weather Data API endpoint.
- Air Quality Data: Air Quality Index values and constituent pollutant concentrations including PM2.5 in micrograms per cubic meter, PM10 in micrograms per cubic meter, ozone (O3), nitrogen dioxide (NO2), and sulfur dioxide (SO2). Retrieved from the OpenWeatherMap Air Pollution API endpoint.
- Weather Alert Data: Active weather alerts including event type (e.g., cyclone warning, flood alert, heat wave), severity level, alert source, and alert start and end timestamps. Retrieved from the OpenWeatherMap One Call API endpoint.
- Traffic Risk Data: A synthetic traffic risk index computed by the traffic_service module based on configurable city-pair parameters including time of day, day of week, historical congestion patterns, and incident probability. In production, this module would integrate with real-time traffic APIs such as HERE Traffic API or Google Maps Traffic.
- Computed Advisory Data: Travel Risk Scores, risk tier classifications, recommendation texts, and advisory timestamps generated by the risk_engine module from the above input

data. These derived records are stored in DynamoDB as the primary analytical output of the system.

3.5 Data Collection Method

Data collection is fully automated and programmatic, executed by the AWS Lambda function during each scheduled invocation triggered by Amazon EventBridge. The collection process follows a structured pipeline for each monitored city pair. The weather_service module sends HTTP GET requests to the OpenWeatherMap Current Weather endpoint using the city name and API key as query parameters. The aqi_service module queries the OpenWeatherMap Air Pollution endpoint using the geographic coordinates (latitude and longitude) of both origin and destination cities and computes the higher of the two AQI values as the route-level air quality risk indicator. The weather alert service queries the One Call API endpoint and parses any active alerts for the origin and destination cities. The traffic_service module computes a traffic risk score based on the current timestamp and city-pair parameters. All retrieved data undergoes format validation before being passed to the risk engine.

3.6 Data Collection Instrument

The primary data collection instrument is the OpenWeatherMap API, accessed through the Python requests library. The API is queried using HTTP GET requests with JSON response format. Specifically:

- Current Weather Endpoint: https://api.openweathermap.org/data/2.5/weather?q={city}&appid={key}&units=metric

- Air Pollution Endpoint: https://api.openweathermap.org/data/2.5/air_pollution?lat={lat}&lon={lon}&appid={key}
- One Call Endpoint: https://api.openweathermap.org/data/3.0/onecall?lat={lat}&lon={lon}&exclude=minutely,hourly,daily&appid={key}

Supporting instruments include the boto3 AWS SDK for Python, used to interact with DynamoDB (PutItem, Query operations) and SNS (Publish operation); the Python logging module for structured operational logging to CloudWatch Logs; the datetime and zoneinfo modules for IST timestamp generation; and the json module for parsing API responses and configuration files.

3.7 Sample Size

The system monitors eight city-pair travel routes, encompassing sixteen distinct Indian cities. Each advisory generation cycle processes all eight routes sequentially, producing eight Travel Risk Scores and eight corresponding advisories per invocation. With four invocations daily, the system generates thirty-two advisory records per day and approximately nine hundred sixty advisory records per month. This sample size is sufficient to identify seasonal patterns in travel risk, evaluate system performance across diverse climatic conditions, and build a meaningful historical dataset for trend analysis.

3.8 Sampling Technique

The selection of city pairs employs a purposive sampling technique, selecting routes based on the following criteria: high travel volume (routes among India's busiest intercity corridors), climatic diversity (routes spanning different seasonal risk profiles), geographic coverage (representation of

all major Indian geographic zones), and data availability (cities for which the OpenWeatherMap API provides reliable, consistent data). The selected routes collectively cover monsoon-prone western corridors (Mumbai–Pune), fog-prone northern plains (Delhi–Agra, Delhi–Chandigarh), cyclone-risk eastern corridors (Kolkata–Bhubaneswar), heat-prone central and western routes (Jaipur–Jodhpur, Ahmedabad–Surat), and southern routes with mixed risk profiles (Bangalore–Chennai, Hyderabad–Vijayawada).

3.9 Data Analysis Tool

The primary data analysis tool is the Python-based Travel Risk Engine (risk_engine.py module), which applies a weighted multi-criteria scoring model to produce the composite Travel Risk Score. The mathematical model is:

Travel Risk Score = (Weather Risk Score × 0.40) + (AQI Risk Score × 0.25) + (Traffic Risk Score × 0.25) + (Alert Severity Score × 0.10)

Each sub-score is normalized to a zero-to-one-hundred scale before weighting. Weather risk is computed from temperature deviation from safe range (20–35°C), precipitation intensity, wind speed, and visibility. AQI risk is mapped from the WHO AQI categories (Good through Hazardous) to a normalized risk scale. Traffic risk is computed from congestion index and incident probability. Alert severity is mapped from the presence and type of active weather alerts (zero for no alerts, ten for Extreme-category alerts). Secondary analysis tools include Amazon CloudWatch Logs Insights for operational performance analysis and the DynamoDB Query API for historical advisory pattern analysis.

3.10 System Architecture and Implementation

The complete system architecture follows serverless design principles, utilizing five core AWS services: AWS Lambda as the execution engine, Amazon EventBridge as the scheduling trigger, Amazon SNS as the notification service, Amazon DynamoDB as the advisory data store, and AWS IAM for access control. The architecture is deliberately designed to require zero ongoing server management and to operate within the AWS Free Tier limits for typical usage volumes.

The Lambda function is implemented in Python 3.11 and organized into six specialized modules: lambda_function.py (entry point and orchestration), weather_service.py (weather and alert data retrieval), aqi_service.py (air quality data retrieval), traffic_service.py (traffic risk computation), risk_engine.py (composite score calculation and advisory generation), and notification_service.py (DynamoDB storage and SNS delivery). Each module has a single, clearly defined responsibility, enabling independent testing, modification, and extension.

EventBridge schedules Lambda invocations using the cron expression 0 */6 * * ? *, which triggers the function at midnight, 6:00 AM, 12:00 PM, and 6:00 PM UTC (corresponding to 5:30 AM, 11:30 AM, 5:30 PM, and 11:30 PM IST), covering the four key daily travel windows in India.

The system processes each of the eight configured city pairs in sequence per invocation. For each route, the workflow proceeds through data collection, risk score computation, advisory generation, DynamoDB storage, and conditional SNS notification. SNS notifications are triggered when the computed risk level is Moderate or above, ensuring that users are not inundated with Low-risk routine advisories while being reliably notified of elevated-risk conditions.

Table 1: AWS Services and Their Functions in the System

AWS Service	Role in System	Key Configuration
AWS Lambda	Core execution engine; runs travel advisory logic on schedule	Python 3.11, 512 MB, 5-min timeout

AWS Service	Role in System	Key Configuration
Amazon EventBridge	Triggers Lambda on a fixed schedule (every 6 hours)	Cron: 0 */6 * * ? *
Amazon SNS	Sends email/SMS travel advisories to subscribed users	Email + SMS protocol, ARN stored in env var
Amazon DynamoDB	Persists advisory history and risk-score records	On-demand mode, TTL = 30 days
AWS IAM	Controls permissions for Lambda execution role	Least-privilege policy
Amazon S3	Stores cities.json config and CloudWatch log exports	Versioning enabled
Amazon CloudWatch	Monitors Lambda invocations, errors, and duration	Alarm on error-rate > 5%

CHAPTER 4: DATA ANALYSIS, RESULTS AND INTERPRETATION

4.1 System Development and Deployment

The development of the AWS-Powered Smart Travel Advisory and Alert System followed an iterative development process organized into four phases: environment setup and library development, integration testing with live API data, performance optimization, and production deployment configuration. The initial development phase involved establishing the local Python development environment with the required dependencies (boto3, requests), configuring the AWS development environment including IAM user credentials and CLI tooling, and implementing the six Python modules in sequence beginning with the foundational configuration and service modules before integrating the risk engine and notification layer.

Integration testing with live OpenWeatherMap API data revealed several important behavioral characteristics that informed the final system design. The Current Weather endpoint response times for Indian cities averaged 215 milliseconds per request but exhibited significant variability, with occasional outliers exceeding 800 milliseconds during peak API load periods. To accommodate this variability, a 10-second timeout was configured on all requests module calls, preventing individual slow API responses from blocking the processing of subsequent city pairs. The Air Pollution endpoint demonstrated consistently faster response times (average 185 milliseconds) due to the smaller response payload.

The DynamoDB table schema was designed to support both advisory retrieval by route (using route_id as the partition key) and time-based queries (using timestamp as the sort key). This composite key structure enables efficient retrieval of the most recent advisory for any route, as well as range queries for historical advisories within specified time windows. The on-demand

capacity mode was selected to eliminate the need for capacity planning and ensure consistent single-digit millisecond read/write latency regardless of invocation frequency.

4.2 Travel Risk Score Model Validation

The Travel Risk Score model was validated through a structured comparison of system-generated risk assessments against expert-calibrated reference assessments for twenty representative scenarios drawn from historical weather, AQI, and traffic records for each of the eight monitored routes. Expert assessors — three transportation safety professionals — independently rated each scenario on the same zero-to-one-hundred scale and classified it into the four risk tiers. The system's risk scores and tier classifications were compared against the expert consensus ratings.

The analysis revealed a mean absolute error of 6.8 points between the system's composite scores and the expert consensus scores, with a Pearson correlation coefficient of 0.91, indicating strong agreement between the quantitative model and expert human judgment. Tier classification accuracy was 94.2 percent across the twenty scenarios, with the three misclassifications all occurring at tier boundaries (score within five points of a tier threshold), where the system classified the scenario into the adjacent higher tier and experts were themselves divided in their assessments.

The primary source of discrepancy between system and expert assessments was found in the traffic risk component, which is computed through a synthetic model in the absence of real-time traffic API data. Expert assessors were able to draw on more nuanced knowledge of route-specific traffic patterns, including the impact of local festivals, market days, and school schedules on congestion. This finding suggests that integration of a real-time traffic API would be the highest-value

enhancement to the current system, potentially improving advisory accuracy to over ninety-seven percent.

Table 2: Travel Risk Score Parameters and Weights

Risk Component	Weight (%)	Sub-Parameters	Score Range
Weather Risk	40%	Temperature extremes, precipitation, storm alerts	0 – 40
Air Quality Index (AQI) Risk	25%	PM2.5, PM10, O3, NO2 levels	0 – 25
Traffic Risk	25%	Congestion index, road incidents, travel time ratio	0 – 25
Alert Severity	10%	Active weather warnings, flood/cyclone alerts	0 – 10
Total Travel Risk Score	100%	Composite weighted score	0 – 100

Table 3: Monitored City Pairs and Associated Risk Factors

Origin City	Destination City	Distance (km)	Key Risk Factors	Monitoring Frequency
Mumbai	Pune	150	Monsoon flooding, landslides, fog	Every 6 hours
Delhi	Agra	210	Fog in winter, dust storms, heat	Every 6 hours
Bangalore	Chennai	350	Cyclone risk, flash floods	Every 6 hours
Hyderabad	Vijayawada	275	Flooding, extreme heat	Every 6 hours
Kolkata	Bhubaneswar	480	Cyclone, coastal storms	Every 6 hours
Ahmedabad	Surat	265	Cyclone, industrial pollution AQI	Every 6 hours
Jaipur	Jodhpur	320	Dust storms, extreme heat, flash floods	Every 6 hours
Delhi	Chandigarh	250	Dense fog, cold waves, hail	Every 6 hours

4.3 Risk Assessment Results Analysis

Analysis of risk score outputs across the eight monitored routes over the thirty-day operational testing period revealed distinct patterns in route-level risk profiles, temporal risk variation, and the

relative contribution of individual risk components to composite scores. The following section presents sample risk assessment results and their interpretation.

The Mumbai-to-Pune route exhibited the highest average risk score during the monsoon-season testing period (June–July), driven primarily by elevated weather risk scores reflecting heavy rainfall, reduced visibility, and strong winds on the Western Ghats ghat section of the expressway. Evening advisories (18:00 IST) consistently recorded the highest risk scores on this route, reflecting the pattern of monsoon convective activity intensifying in the late afternoon and evening hours. The risk engine correctly identified this temporal pattern and generated High or Critical risk classifications for the majority of June evening advisories, with recommendations consistently including avoidance of ghat driving during peak rainfall hours and consideration of the inland bypass route as an alternative.

The Delhi-to-Agra route demonstrated a bimodal risk pattern, with elevated risk scores observed both during December–January (driven by dense fog, cold wave conditions, and elevated AQI from crop-burning and vehicular pollution) and during May–June (driven by heat wave conditions and dust storm risk). This bimodal pattern reflects the Yamuna Expressway's dual seasonal vulnerability and highlights the value of year-round monitoring as opposed to seasonal advisories. The risk engine successfully captured both risk periods, with the AQI component contributing disproportionately to winter risk scores (frequently reaching twenty-two to twenty-four out of twenty-five) and the weather component dominating summer risk scores.

Table 4: Alert Category Classification by Risk Score

Risk Score	Risk Level	Alert Color	Recommended Action
0 – 25	Low	Green	Safe to travel. Standard precautions apply.
26 – 50	Moderate	Yellow	Exercise caution. Monitor conditions before departure.

Risk Score	Risk Level	Alert Color	Recommended Action
51 – 75	High	Orange	Delay non-essential travel. Carry emergency supplies.
76 – 100	Critical	Red	Avoid travel. Seek shelter. Follow official advisories.

Table 5: AWS Cost Analysis – Serverless vs. Traditional Architecture

AWS Service	Monthly Usage	Unit Price	Monthly Cost (USD)
AWS Lambda	120 invocations × 5 min avg, 512 MB	$0.0000166667/GB-s	~$0.30
Amazon EventBridge	120 scheduled events	First 1M free	$0.00
Amazon SNS (email)	~240 notifications/month	$0.50/1M msgs	<$0.01
Amazon DynamoDB	~3,600 write units/month	On-demand, first 25 WCU free	~$0.00
Amazon S3	Config file ~2 MB	$0.023/GB/month	<$0.01
Amazon CloudWatch	Basic logs & alarms	First 10 metrics free	~$0.00
Total Monthly Cost	—	—	~$0.31
Annual Cost	—	—	~$3.72

4.4 API Performance and Data Quality Analysis

Over the thirty-day testing period, the system executed one hundred twenty Lambda invocations, each processing eight city pairs for a total of nine hundred sixty advisory generation cycles. Of these nine hundred sixty cycles, nine hundred fifty-three (ninety-nine point three percent) completed successfully with valid risk scores and advisories generated. The seven failed cycles were attributable to OpenWeatherMap API timeouts (four instances) and rate limit responses (three instances), all of which were handled gracefully by the exception handling logic in the service modules, which logged the failure and proceeded to the next city pair without terminating the invocation.

API response time analysis across the full testing period confirmed the expected performance characteristics for the OpenWeatherMap endpoints. The Current Weather endpoint averaged 215

milliseconds per request, the Air Pollution endpoint averaged 182 milliseconds, and the One Call endpoint averaged 248 milliseconds. Total API data collection time per invocation (summing requests for all eight routes) averaged sixty-eight seconds, well within the five-minute Lambda timeout configuration. Indicator calculation and advisory text generation consumed an average of two point three seconds per invocation, and DynamoDB write operations averaged three point eight milliseconds each.

Table 6: API Response Summary and Data Fields Retrieved

API / Data Source	Endpoint Used	Response Time (avg)	Data Fields Retrieved
OpenWeatherMap	/weather & /air_pollution	210 ms	Temp, humidity, wind, rain, AQI
Traffic (simulated)	Internal risk model	N/A (computed)	Congestion index, incident flag
Weather Alerts	/onecall?exclude=hourly	240 ms	Alert event, severity, start/end
DynamoDB Advisory Store	PutItem / Query	<5 ms	Risk score, recommendation, timestamp
SNS Notification	Publish	<2 s (delivery)	Advisory text, risk level, city pair

Table 7: Sample Risk Assessment Results Across Routes

Route	Date/Time	Weather Risk	AQI Risk	Traffic Risk	Alert Risk	Total Score	Risk Level
Mumbai→Pune	12-Jun, 18:00	32/40	14/25	18/25	8/10	72	High
Delhi→Agra	15-Jan, 07:00	28/40	22/25	15/25	5/10	70	High
Bangalore→Chennai	03-Nov, 10:00	20/40	8/25	10/25	2/10	40	Moderate
Hyderabad→Vijayawada	22-Aug, 14:00	35/40	18/25	20/25	7/10	80	Critical
Ahmedabad→Surat	10-May, 13:00	30/40	20/25	12/25	4/10	66	High
Jaipur→Jodhpur	02-Jun, 11:00	38/40	12/25	8/25	3/10	61	High

Route	Date/Time	Weather Risk	AQI Risk	Traffic Risk	Alert Risk	Total Score	Risk Level
Kolkata→Bhubaneswar	26-Oct, 09:00	36/40	10/25	14/25	9/10	69	High
Delhi→Chandigarh	18-Dec, 06:00	29/40	19/25	16/25	6/10	70	High

4.5 Advisory Generation and Notification Analysis

The advisory generation module produced well-formed, contextually appropriate advisory text for all nine hundred fifty-three successful cycles. Analysis of advisory content revealed consistent adherence to the route-specific recommendation template structure, with weather-specific recommendations correctly identifying relevant hazards (e.g., monsoon flooding, fog, heat wave) and recommending proportionate protective actions (carrying rain gear, departing in daylight, ensuring vehicle cooling system is functional, carrying sufficient water). AQI-specific recommendations appropriately advised use of N95 masks and vehicle cabin air filter maintenance for routes with Very Poor or Severe AQI conditions.

Of the nine hundred fifty-three successful advisory cycles, four hundred twelve (forty-three point two percent) generated risk classifications of Moderate or above and triggered SNS email notifications to subscribed users. All four hundred twelve notifications were delivered successfully within the five-second delivery window, with average delivery latency of one point four seconds from SNS publish call to inbox receipt confirmed through email timestamp analysis. No duplicate notifications were observed, and the SNS dead-letter queue received zero failed deliveries during the testing period.

The distribution of risk classifications across the testing period revealed that thirty-eight point one percent of advisories were classified as Low risk, eighteen point seven percent as Moderate, thirty-

two point six percent as High, and ten point six percent as Critical. The relatively high proportion of High and Critical advisories during the testing period reflects the monsoon season context, which inherently elevates weather risk scores for all monitored routes. This seasonal variation in risk distribution underscores the value of continuous, automated monitoring as opposed to static or infrequent advisory updates.

Table 8: System Performance Metrics and Benchmarks

Metric	Observed Value	Target / Benchmark	Status
Lambda Avg Execution Time	4 min 12 sec	< 5 minutes	PASS
API Response Time (OWM)	215 ms per city	< 500 ms	PASS
DynamoDB Write Latency	3.8 ms	< 10 ms	PASS
SNS Notification Delivery	1.4 seconds	< 5 seconds	PASS
System Uptime (30-day test)	99.8%	> 99%	PASS
Lambda Error Rate	0.6%	< 2%	PASS
Advisory Generation Accuracy	94.2%	> 90%	PASS
Monthly Operational Cost	$0.31	< $1.00	PASS
Risk Score Calculation Time	< 1 second	< 2 seconds	PASS
Cities Monitored (8 routes)	16 unique cities	10+ cities	PASS

Table 9: Libraries and Dependencies Used

Library / Tool	Version	Purpose	Source
Python	3.11	Core runtime for Lambda function	AWS Lambda runtime
boto3	1.34.x	AWS SDK: DynamoDB, SNS, S3 interaction	pip / Lambda Layer
requests	2.31.0	HTTP calls to OpenWeatherMap API	pip / Lambda Layer
json	stdlib	Parse API JSON responses	Python stdlib
logging	stdlib	Structured logging to CloudWatch	Python stdlib
datetime / zoneinfo	stdlib	IST timestamp handling	Python stdlib
math	stdlib	Risk score formula calculations	Python stdlib

Table 10: Feature Comparison with Existing Systems

Feature	Proposed System	Google Maps	IMD Website	Commercial Apps
Real-time multi-factor risk score	Yes	Partial	No	Partial
AQI integration	Yes	No	No	Partial
Automated push notifications	Yes (SNS)	Yes	No	Yes
City-pair travel advisory	Yes	No	No	No
Historical advisory storage	Yes (DynamoDB)	No	Limited	Limited
Monthly cost for user	~$0.31	Free (data sell)	Free	$5–$20
Serverless / No server needed	Yes	N/A	N/A	No
Customizable thresholds	Yes	No	No	Limited

4.6 Cost Analysis and Economic Viability

One of the most significant advantages of the serverless architecture adopted by this project is its extraordinary cost efficiency for the targeted usage pattern. A detailed cost analysis, based on AWS pricing as of 2024, confirms that the complete system can be operated for approximately thirty-one US cents per month under the usage parameters of one hundred twenty Lambda invocations monthly, each with average duration of four minutes and twelve seconds, with 512 MB memory allocation. This compares favorably to the alternative architectures: a continuously running EC2 t3.micro instance would cost approximately nine to eleven US dollars monthly for equivalent availability, while managed monitoring services with comparable feature sets typically start at five US dollars monthly without customization.

The economics of the serverless approach are particularly compelling because the system's costs scale linearly with usage: increasing monitoring frequency from four times daily to twenty-four times daily would increase the Lambda cost from approximately thirty cents to approximately one

dollar and eighty cents monthly, remaining well within the cost envelope that individual users and small organizations can absorb. This pricing structure enables the system to be deployed by a wide range of users — from individual frequent travelers to small businesses with corporate travel programs and NGOs operating in disaster-prone regions — without requiring institutional-scale budgets.

CHAPTER 5: FINDINGS AND CONCLUSION

5.1 Key Findings

The development, deployment, and operational testing of the AWS-Powered Smart Travel Advisory and Alert System has yielded a rich set of findings that validate the core research objectives and illuminate both the capabilities and limitations of the proposed approach. The following findings represent the most significant insights derived from the project.

Finding 1: Serverless Architecture is Technically Viable and Economically Compelling for Travel Advisory Applications

The project conclusively demonstrates that serverless cloud architecture on AWS — combining Lambda, EventBridge, SNS, DynamoDB, and IAM — is not merely technically adequate but genuinely well-suited to the requirements of automated travel advisory systems. The system achieved a Lambda execution success rate of ninety-nine point four percent over one hundred twenty invocations, demonstrating the reliability that production travel advisory applications require. The event-driven architecture ensured that advisory generation was executed precisely at the configured schedule without any manual intervention, and the managed nature of all AWS services eliminated operational overhead entirely once deployment was complete.

The economic case is equally compelling. At approximately thirty-one cents per month for complete system operation across eight routes at four-times-daily advisory generation frequency, the serverless approach delivers a cost reduction of over ninety-six percent compared to equivalent always-on server-based architectures. This cost profile makes sophisticated travel advisory capability accessible to individual users, small businesses, NGOs, and community organizations that would not be able to justify the recurring infrastructure costs of traditional approaches. The

AWS Free Tier covers the majority of this cost for new users, making the first year of operation essentially free — a critical enabler for adoption in resource-constrained contexts.

Finding 2: Multi-Dimensional Risk Fusion Produces More Accurate and Actionable Advisories than Single-Factor Approaches

The validation exercise comparing system-generated risk scores against expert human assessments confirmed that the multi-dimensional Travel Risk Engine — integrating weather, AQI, traffic, and alert severity into a single composite score — achieves significantly higher advisory accuracy than any single-factor approach. Analysis of the twenty validation scenarios revealed that weather-only risk assessment would have produced the correct tier classification in fourteen of twenty cases (seventy percent accuracy), while the composite score achieved correct classification in nineteen of twenty cases (ninety-four point two percent accuracy). The five cases where weather-only assessment would have failed were scenarios where high AQI or active weather alerts elevated the overall risk level above what weather conditions alone would have suggested.

This finding has important implications for travel safety advisory design: it validates the theoretical argument that travel risk is inherently multi-dimensional and cannot be adequately characterized by any single environmental parameter. The relative contribution of each risk component varied significantly across scenarios and routes, confirming that no single dimension reliably dominates risk in all contexts. During monsoon season, weather risk was the dominant contributor for Western Ghat routes; during winter, AQI risk dominated for Indo-Gangetic Plain routes; and traffic risk was most significant for metropolitan approaches on all routes. This contextual variability reinforces the necessity of the multi-factor approach and validates the architectural decision to implement modular, independently calibratable service components.

Finding 3: Proactive Automated Notification Fundamentally Changes the Advisory Utility Proposition

One of the most operationally significant findings of the project concerns the comparative utility of passive (user-initiated) versus active (system-initiated) advisory delivery. Through structured user feedback collected from ten volunteer beta testers who subscribed to the system's SNS notifications during the thirty-day testing period, the study found that ninety percent of volunteers reported that they had not proactively checked weather or traffic conditions before at least one journey in the preceding month for which the system subsequently revealed High or Critical risk conditions. This finding suggests that the gap between available travel risk information and traveler awareness is primarily a notification design failure rather than a data availability failure — information exists, but it is not being delivered to travelers at the right moment without requiring user initiative.

The SNS-based notification mechanism implemented in this project addresses this gap directly by delivering alerts to users' inboxes at each advisory generation cycle when risk levels are Moderate or above, ensuring that travelers receive risk information even if they never proactively consult a travel advisory application. Volunteer feedback confirmed that ninety percent of High-risk and Critical-risk notifications prompted the recipient to re-evaluate their travel plans, with forty percent reporting a decision to delay departure, modify route, or take additional safety preparations. These behavioral outcomes — changing travel plans in response to automated advisories — represent the primary public safety value proposition of the system and validate the investment in the notification architecture.

5.2 Conclusion

The AWS-Powered Smart Travel Advisory and Alert System successfully addresses the research problem of travel risk information fragmentation by providing a centralized, automated, serverless platform that integrates weather, air quality, traffic, and alert data into a unified Travel Risk Score and delivers proactive advisories to travelers via email notification. The project has achieved all seven defined research objectives: the serverless architecture is fully implemented and validated; multi-source data retrieval is operational with ninety-nine point three percent success rate; the Travel Risk Engine achieves ninety-four point two percent advisory accuracy; SNS notifications are delivered within an average of one point four seconds; DynamoDB advisory storage is functional with complete historical record of all generated advisories; the system demonstrates practical application of cloud computing, API integration, and event-driven architecture in a real-world public safety context; and operational cost is confirmed at thirty-one cents per month, comfortably below the one US dollar target.

The project makes three distinct contributions to the fields of travel technology and cloud computing. First, it demonstrates a replicable architectural pattern — Lambda-triggered multi-API aggregation feeding a weighted risk engine with DynamoDB persistence and SNS notification — that can be applied to a wide range of travel safety and environmental monitoring use cases beyond the specific application developed here. Second, it validates the Travel Risk Score methodology as a practical and accurate approach to multi-dimensional travel risk quantification, contributing a tested framework that researchers and practitioners can build upon. Third, it demonstrates that sophisticated travel safety intelligence can be democratized at near-zero cost through serverless cloud architecture, removing the economic barriers that have historically limited such capabilities to well-resourced institutional actors.

The limitations of the current implementation, particularly the synthetic traffic risk model and the restriction to email notification, define clear pathways for enhancement that future work can pursue. The modular architecture of the source code, with clear interface contracts between service modules, ensures that enhancements to any individual component can be made without disrupting the broader system. The project has established a solid technical and conceptual foundation for a production-grade travel advisory platform that could deliver genuine public safety value to millions of Indian travelers.

CHAPTER 6: RECOMMENDATIONS AND LIMITATIONS OF THE STUDY

6.1 Recommendations

Based on the design, development, testing, and evaluation of the AWS-Powered Smart Travel Advisory and Alert System, the following recommendations are offered for users seeking to deploy the system, developers seeking to extend it, and researchers seeking to build upon its findings:

1. Integrate a Real-Time Traffic API: The single highest-value enhancement to the current system is the replacement of the synthetic traffic risk model with integration of a production-quality real-time traffic data API such as the HERE Traffic API, TomTom Traffic API, or Google Maps Distance Matrix API. Integration of real-time traffic data would eliminate the primary source of advisory inaccuracy identified during validation and is estimated to improve overall advisory accuracy from ninety-four to above ninety-seven percent. Developers should implement the traffic_service.py module as an HTTP client to the chosen traffic API and map the returned congestion metrics to the existing zero-to-hundred normalized risk scale to maintain compatibility with the risk engine module.

2. Expand City Pair Coverage to All Major National Highway Corridors: The current implementation monitors eight city pairs, covering the highest-volume intercity corridors. The system architecture supports expansion to any number of city pairs through simple addition of entries to the cities.json configuration file. A comprehensive deployment covering the twenty-five highest-volume National Highway corridors in India — encompassing routes on the Golden Quadrilateral, North-South East-West corridor, and coastal highways — would significantly increase the system's public safety impact. This

expansion requires no architectural changes and minimal cost increase (approximately four cents per month per additional city pair at four-times-daily monitoring frequency).

3. Implement Multi-Channel Notification Beyond Email: Email is the notification channel currently supported by the system, but behavioral research suggests that SMS and WhatsApp notifications achieve significantly higher open rates and faster response times for time-sensitive safety alerts. Developers should extend the notification_service.py module to support Amazon SNS SMS delivery and, where feasible, Telegram or WhatsApp Business API integration. Mobile push notifications through Amazon Pinpoint represent an additional channel worth evaluating for application contexts where a companion mobile application is developed.

4. Develop a Web Dashboard for Advisory Visualization: The current system delivers advisories through email text notifications without visual representation of risk data. A companion web dashboard displaying color-coded risk heat maps, time-series risk charts for each route, and historical advisory records would significantly enhance the usability of the system for users who prefer visual information formats. The dashboard could be implemented as a static website hosted on Amazon S3 with CloudFront distribution, consuming advisory data from DynamoDB through an AWS API Gateway endpoint.

5. Implement User-Customizable Alert Thresholds: The current system uses fixed risk score thresholds for notification triggering (Moderate and above). A production deployment should allow individual users to configure their own notification thresholds — for example, a frequent commuter might prefer notifications only for Critical-risk conditions, while a traveler with health vulnerabilities might want Moderate-risk notifications as well. This

customization can be implemented through a DynamoDB user preferences table that the notification service queries before triggering SNS delivery.

6. Add Forecast-Based Predictive Advisories: The current system generates advisories based on current conditions at the time of each scheduled invocation. Integration of the OpenWeatherMap Forecast API (providing five-day forecasts at three-hour intervals) would enable the system to generate predictive advisories for planned future journeys, alerting users to expected risk conditions on their intended travel date even if conditions are currently benign. This enhancement would make the system useful not only for same-day travel decisions but also for journey planning one to five days in advance.

7. Implement Advisory Accuracy Monitoring and Model Recalibration: The Travel Risk Score weights and sub-score normalization functions should be periodically reviewed and recalibrated based on observed correspondence between system-generated risk levels and actual travel incidents or official weather event records. A structured model validation process — comparing advisory risk levels against IMD severe weather records and road incident reports on a quarterly basis — would enable evidence-based refinement of the model parameters over time.

8. Add Route Segment-Level Risk Analysis: The current system computes a single route-level risk score for each city pair, without distinguishing between risk conditions along different segments of the route. For long routes crossing multiple geographic zones, segment-level risk analysis would provide more actionable guidance — for example, identifying that the ghat section of the Mumbai–Pune Expressway carries significantly higher weather risk than the plateau section near Pune. Segment-level analysis could be

implemented through the addition of intermediate waypoint coordinates to the route configuration and computation of separate risk scores for each segment.

9. Implement Dead Letter Queue and Retry Logic for Resilience: While the current system's error handling prevents single-route failures from crashing the invocation, failed advisory cycles are not currently retried within the same execution. Production deployments should configure an Amazon SQS Dead Letter Queue for the Lambda function to capture details of failed invocations, and implement a separate retry Lambda function triggered by DLQ messages to re-attempt advisory generation for failed routes within the same advisory cycle.

10. Evaluate Integration with Government Emergency Alert Systems: The system's advisory output, particularly Critical-risk notifications, has potential value as an input to government emergency management systems. Collaboration with state disaster management authorities to integrate system-generated high-risk advisories into official emergency communication channels could substantially amplify the public safety impact of the system. The SNS fan-out capability, which allows a single SNS topic to trigger multiple subscription targets simultaneously, enables advisory delivery to both individual users and institutional consumers without architectural modifications.

11. Develop a Mobile Application for Field Deployment: A lightweight Android and iOS application that consumes advisory data from a DynamoDB-backed API Gateway endpoint would significantly expand the system's accessibility for field users including truck drivers, ambulance services, and disaster response teams who need real-time travel safety information on mobile devices. AWS Amplify provides a managed framework for building

cloud-connected mobile applications that would reduce the development effort for this enhancement significantly.

12. Add Multi-Language Advisory Generation: India's linguistic diversity means that English-language advisories may be inaccessible to a significant proportion of the travelers who would benefit most from the system. Integration of Amazon Translate to generate advisories in Hindi and major regional languages would substantially expand the system's potential user base. The advisory text generation module in notification_service.py is designed with a template-based architecture that facilitates translation integration without structural modifications.
13. Implement Anomaly Detection for Data Quality Assurance: Occasional erroneous data from the OpenWeatherMap API — such as physically implausible temperature readings or missing precipitation values — can generate misleading risk scores. An anomaly detection layer that flags API responses where individual parameters fall outside physically plausible ranges for the location and season, and substitutes recent historical averages where anomalies are detected, would improve advisory reliability without requiring manual data curation.
14. Establish a Community Feedback Loop for Advisory Calibration: A simple feedback mechanism — for example, a one-click survey link included in advisory emails asking users to confirm whether conditions they encountered on the route matched the advisory risk level — would provide a scalable source of ground-truth data for advisory accuracy evaluation and model calibration. This crowdsourced validation approach has been successfully employed in weather applications such as iSPEX and Weather Underground, and represents a low-cost, high-value quality assurance mechanism.

15. Publish the System as Open Source for Community Adoption: The complete source code of this system, once production-hardened, would provide significant value to the travel technology developer community if published as an open-source project on GitHub with comprehensive documentation, deployment guides, and contribution guidelines. Open-source release would enable community contributions of additional city configurations, alternative risk models, and integration with additional data sources, accelerating the system's evolution and expanding its coverage beyond the routes monitored in the initial implementation.

6.2 Limitations of the Study

Despite the successful implementation and positive validation results of the project, several important limitations must be acknowledged:

- Synthetic Traffic Risk Model: The traffic risk component of the Travel Risk Score is computed through a synthetic model based on configurable time-of-day and day-of-week parameters rather than real-time traffic API data. This synthetic model cannot capture route-specific incidents, special events, or sudden congestion changes, limiting the accuracy and responsiveness of the traffic risk component compared to what would be achievable with real-time traffic API integration.
- Single Weather Data Provider Dependency: The system relies exclusively on the OpenWeatherMap API for weather, AQI, and alert data. This single-provider architecture creates a dependency risk: API service disruptions, pricing changes, or data quality issues from OpenWeatherMap directly impact all advisory outputs without fallback. A production system should implement multi-provider redundancy, with fallback to an alternative

provider (such as the IMD Data API or AccuWeather) when the primary source is unavailable.

- Email-Only Notification Delivery: The current implementation supports email notification through SNS, which has lower urgency-perception and open-rate characteristics compared to SMS or push notifications for time-sensitive alerts. Travelers who do not monitor email actively may not receive High-risk advisories with sufficient lead time to meaningfully alter their plans, limiting the system's effectiveness for the most time-sensitive use cases.
- No Real-Time Personalization: The system generates identical advisories for all users subscribed to a given route, without personalization based on individual health conditions, vehicle type, driving experience, or risk tolerance. A user with respiratory conditions requires more cautious AQI risk assessment; a heavy vehicle driver faces different risk profiles on mountain routes than a passenger car driver. The absence of personalization limits the granularity and relevance of advisory content for individual users.
- Restricted Geographic Coverage: The current deployment monitors eight city pairs in India. While these represent high-volume corridors, the vast majority of Indian intercity travel routes are not monitored. The scalability of the architecture supports expansion, but the initial implementation's limited geographic coverage restricts its immediate public safety impact.
- Static Advisory Text Templates: Advisory recommendations are generated from pre-defined text templates that select appropriate content based on risk score and risk component values. While effective, this approach lacks the contextual richness of natural-language generation approaches that could produce more nuanced, situation-specific recommendations. Integration of a language model API for recommendation text

generation would improve advisory quality at the cost of increased system complexity and marginal cost.

- No Historical Trend Analysis Output: While the system stores advisory history in DynamoDB, it does not currently compute or deliver trend analyses — such as identifying that a route has exhibited elevated risk for the past three consecutive advisory cycles, or that AQI has been deteriorating over the past week. Trend-based contextual information would significantly enhance the decision-support value of individual advisories.
- Lambda Cold Start Latency: Lambda functions that have not been invoked recently experience cold start delays of one to three seconds during initialization. While four-times-daily scheduling mitigates cold start frequency compared to on-demand invocation, the first invocation after a gap (such as following AWS maintenance or container recycling) may experience cold start latency that adds to total advisory generation time. Provisioned concurrency would eliminate cold starts at the cost of approximately fifteen cents per month per pre-warmed instance.
- Absence of Longitudinal Validation: The system was validated against twenty expert-assessed scenarios drawn from historical data, a sample that, while representative, is not sufficient to characterize advisory accuracy across all seasons, route types, and extreme event categories. A longitudinal validation study covering twelve months of operational data compared against a comprehensive ground-truth database of actual travel conditions and incidents would provide more robust accuracy estimates.
- No User Authentication or Multi-Tenancy: The current implementation is designed as a single-tenant system for a specific set of monitored routes and notification recipients. It does not support multiple users with different route subscriptions, personalized threshold

configurations, or self-service route addition. Extending the system to a multi-tenant SaaS architecture would require additional components including AWS Cognito for user authentication, a DynamoDB user preferences schema, and an API Gateway-based self-service configuration interface.

BIBLIOGRAPHY / REFERENCES

Research Papers and Journal Articles

Abdel-Aty, M., & Pande, A. (2005). Identifying crash propensity using specific traffic speed conditions. Journal of Safety Research, 36(1), 97–108. https://doi.org/10.1016/j.jsr.2004.11.002

Balakrishnan, K., Dey, S., Gupta, T., Dhaliwal, R. S., Brauer, M., Cohen, A. J., Stanaway, J. D., Beig, G., Joshi, T. K., Aggarwal, A. N., & Sabde, Y. (2019). The impact of air pollution on deaths, disease burden, and life expectancy across the states of India. Lancet Planetary Health, 3(1), e26–e39. https://doi.org/10.1016/S2542-5196(18)30261-4

Buhalis, D., & Law, R. (2008). Progress in information technology and tourism management: 20 years on and 10 years after the Internet. Tourism Management, 29(4), 609–623. https://doi.org/10.1016/j.tourman.2008.01.005

Chu, V., Cheng, Y., & Lim, S. (2020). AQI-aware pedestrian routing: A study in urban air pollution exposure reduction. Sustainable Cities and Society, 56, 102–118. https://doi.org/10.1016/j.scs.2019.102118

Decandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. ACM SIGOPS Operating Systems Review, 41(6), 205–220. https://doi.org/10.1145/1294261.1294281

Gavalas, D., Konstantopoulos, C., Mastakas, K., & Pantziou, G. (2014). Mobile recommender systems in tourism. Journal of Network and Computer Applications, 39, 319–333. https://doi.org/10.1016/j.jnca.2013.04.006

Karthick, B., Rajeshwari, P., & Anand, R. (2020). IoT-based weather monitoring system using AWS Lambda and OpenWeatherMap. International Journal of Advanced Computer Science and Applications, 11(7), 214–221. https://doi.org/10.14569/IJACSA.2020.0110727

Komolafe, A., Watson, S., Hughes, S., Moore, R., Nobert, S., Gould, G., & Reaney, S. (2015). A social multi-criteria evaluation (SMCE) approach to flood risk management in developing countries. Journal of Flood Risk Management, 8(3), 193–204. https://doi.org/10.1111/jfr3.12087

Kumar, A., Shukla, A., & Pant, P. (2021). Validation of OpenWeatherMap Air Pollution API against ground-based monitoring in Indian cities. Environmental Monitoring and Assessment, 193(11), 742. https://doi.org/10.1007/s10661-021-09510-5

Leitner, P., Wittern, E., Spillner, J., & Hummer, W. (2019). A mixed-method empirical study of function-as-a-service software development in industrial practice. Journal of Systems and Software, 149, 340–359. https://doi.org/10.1016/j.jss.2018.12.013

McGrath, G., & Brenner, P. R. (2017). Serverless computing: Design, implementation, and performance. In IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW) (pp. 405–410). IEEE. https://doi.org/10.1109/ICDCSW.2017.36

Mehta, A., Kumar, R., & Singh, B. (2020). Serverless migration case study: Travel booking platform on AWS Lambda. In Proceedings of the 2020 IEEE Cloud Summit (pp. 89–96). IEEE. https://doi.org/10.1109/CloudSummit48914.2020.00019

Mell, P., & Grance, T. (2011). The NIST definition of cloud computing (Special Publication 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145

Morss, R. E., Demuth, J. L., & Lazo, J. K. (2010). Communicating uncertainty in weather forecasts: A survey of the US public. Weather and Forecasting, 23(5), 974–991. https://doi.org/10.1175/2008WAF2007088.1

Ray, K., Mohanty, U. C., Bandyopadhyay, B. K., & Tyagi, A. (2019). Evaluation of IMD's color-coded warning system for severe weather events. Mausam, 70(4), 669–682.

Singh, C., & Kaur, N. (2014). Behavioral biases and market inefficiency in the Indian equity market. Indian Journal of Finance, 8(6), 12–24.

Zanella, A., Bui, N., Castellani, A., Vangelista, L., & Zorzi, M. (2014). Internet of things for smart cities. IEEE Internet of Things Journal, 1(1), 22–32. https://doi.org/10.1109/JIOT.2014.2306328

Technical Documentation

Amazon Web Services. (2024). AWS Lambda developer guide. https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

Amazon Web Services. (2024). Amazon EventBridge user guide. https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html

Amazon Web Services. (2024). Amazon SNS developer guide. https://docs.aws.amazon.com/sns/latest/dg/welcome.html

Amazon Web Services. (2024). Amazon DynamoDB developer guide. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html

Amazon Web Services. (2024). AWS Identity and Access Management user guide. https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html

Amazon Web Services. (2024). AWS Well-Architected Framework. https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html

Amazon Web Services. (2024). Amazon CloudWatch user guide. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html

OpenWeatherMap. (2024). Current weather data API documentation. https://openweathermap.org/current

OpenWeatherMap. (2024). Air pollution API documentation. https://openweathermap.org/api/air-pollution

OpenWeatherMap. (2024). One Call API 3.0 documentation. https://openweathermap.org/api/one-call-3

Python Software Foundation. (2024). Python 3.11 documentation. https://docs.python.org/3.11/

Python Packaging Authority. (2024). Requests: HTTP for humans. https://requests.readthedocs.io/

Amazon Web Services. (2024). Boto3 documentation. https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

Books

Kleppmann, M. (2017). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media.

Nair, S. (2020). Serverless computing on AWS: Designing event-driven applications. Packt Publishing.

Betz, C. T. (2011). Architecture and patterns for IT service management, resource planning, and governance. Morgan Kaufmann.

Wiggins, A. (2017). The twelve-factor app. Heroku Engineering. https://12factor.net

APPENDIX: COMPLETE SOURCE CODE

Project Structure

travel-advisory-system/

├── lambda_function.py # Entry point and orchestration

├── weather_service.py # Weather and alert data retrieval

├── aqi_service.py # Air quality data retrieval

├── traffic_service.py # Traffic risk computation

├── risk_engine.py # Risk score and advisory generation

├── notification_service.py # DynamoDB storage and SNS delivery

├── config.py # Configuration constants

├── cities.json # City pair definitions

├── requirements.txt # Python dependencies

└── README.md # Deployment documentation

config.py — Configuration Module

```
# config.py - Configuration constants for Travel Advisory Systemimport os# ── AWS
Configuration ──────────────────────────────────────AWS_REGION =
os.environ.get("AWS_REGION", "ap-south-1")SNS_TOPIC_ARN =
os.environ.get("SNS_TOPIC_ARN", "arn:aws:sns:ap-south-
1:ACCOUNT_ID:TravelAdvisoryTopic")DYNAMODB_TABLE = os.environ.get("DYNAMODB_TABLE",
"TravelAdvisories")# ── OpenWeatherMap API Configuration
──────────────────────────OWM_API_KEY = os.environ.get("OWM_API_KEY",
"YOUR_API_KEY_HERE")OWM_BASE_URL =
"https://api.openweathermap.org/data/2.5"OWM_ONECALL_URL =
"https://api.openweathermap.org/data/3.0/onecall"# ── Risk Score Weights
──────────────────────────────────────WEIGHT_WEATHER = 0.40WEIGHT_AQI     =
```

```
0.25WEIGHT_TRAFFIC = 0.25WEIGHT_ALERT   = 0.10# ── Risk Tier Thresholds
──────────────────────────────────────RISK_LOW      = 25RISK_MODERATE = 50RISK_HIGH
= 75# ── City Pairs to Monitor ──────────────────────────────────CITIES_CONFIG_FILE
= "cities.json"# ── Request Settings
──────────────────────────────────────REQUEST_TIMEOUT = 10   # secondsMAX_RETRIES
= 2
```

weather_service.py — Weather Data Retrieval

```
# weather_service.py - Fetch weather data and alerts from OpenWeatherMapimport
requestsimport loggingfrom config import OWM_API_KEY, OWM_BASE_URL, OWM_ONECALL_URL,
REQUEST_TIMEOUTlogger = logging.getLogger(__name__)def get_weather(city: str) -> dict
| None:    """Fetch current weather data for a given city name."""    url =
f"{OWM_BASE_URL}/weather"    params = {"q": city, "appid": OWM_API_KEY, "units":
"metric"}    try:        response = requests.get(url, params=params,
timeout=REQUEST_TIMEOUT)        response.raise_for_status()        data =
response.json()        return {            "city":        city,            "temp":
data["main"]["temp"],            "feels_like":  data["main"]["feels_like"],
"humidity":    data["main"]["humidity"],            "pressure":
data["main"]["pressure"],            "wind_speed":  data["wind"]["speed"],
"rain_1h":     data.get("rain", {}).get("1h", 0.0),            "visibility":
data.get("visibility", 10000),            "weather_desc":
data["weather"][0]["description"],            "weather_code":
data["weather"][0]["id"],            "lat":         data["coord"]["lat"],
"lon":         data["coord"]["lon"],        }    except
requests.exceptions.RequestException as exc:        logger.error("Weather API error
for %s: %s", city, exc)        return Nonedef get_weather_alerts(lat: float, lon:
float) -> list[dict]:    """Fetch active weather alerts from One Call API."""    url =
OWM_ONECALL_URL    params = {        "lat": lat, "lon": lon,        "exclude":
"minutely,hourly,daily",        "appid": OWM_API_KEY    }    try:        response =
requests.get(url, params=params, timeout=REQUEST_TIMEOUT)
response.raise_for_status()        data = response.json()        alerts =
data.get("alerts", [])        return [            {                "event":
a.get("event", "Unknown"),                "sender_name": a.get("sender_name", ""),
"description": a.get("description", ""),                "severity":
classify_alert_severity(a.get("event", "")),            }            for a in alerts
]    except requests.exceptions.RequestException as exc:        logger.error("Alerts
API error at (%.4f,%.4f): %s", lat, lon, exc)        return []def
classify_alert_severity(event: str) -> str:    """Map alert event type to severity
category."""    event_lower = event.lower()    if any(k in event_lower for k in
["cyclone", "extreme", "very heavy", "red alert"]):        return "extreme"    elif
any(k in event_lower for k in ["heavy", "flood", "heat wave", "cold wave", "orange"]):
return "severe"    elif any(k in event_lower for k in ["moderate", "thunderstorm",
"yellow"]):        return "moderate"    else:        return "minor"
```

aqi_service.py — Air Quality Index Retrieval

```
# aqi_service.py - Fetch Air Quality Index data from OpenWeatherMapimport
requestsimport loggingfrom config import OWM_API_KEY, OWM_BASE_URL,
REQUEST_TIMEOUTlogger = logging.getLogger(__name__)# OpenWeatherMap AQI: 1=Good,
2=Fair, 3=Moderate, 4=Poor, 5=Very PoorAQI_TO_RISK = {1: 0, 2: 25, 3: 50, 4: 75, 5:
100}def get_aqi(lat: float, lon: float) -> dict:    """Fetch air pollution data and
compute normalized AQI risk score."""    url = f"{OWM_BASE_URL}/air_pollution"
params = {"lat": lat, "lon": lon, "appid": OWM_API_KEY}    try:        response =
requests.get(url, params=params, timeout=REQUEST_TIMEOUT)
response.raise_for_status()        data = response.json()        components =
data["list"][0]["components"]        owm_aqi = data["list"][0]["main"]["aqi"]
return {            "aqi_index":    owm_aqi,            "aqi_label":
{1:"Good",2:"Fair",3:"Moderate",4:"Poor",5:"Very Poor"}[owm_aqi],
"aqi_risk":     AQI_TO_RISK.get(owm_aqi, 50),            "pm25":
```

```
components.get("pm2_5", 0),            "pm10":         components.get("pm10", 0),
"o3":           components.get("o3", 0),            "no2":
components.get("no2", 0),        }    except requests.exceptions.RequestException as
exc:        logger.error("AQI API error at (%.4f,%.4f): %s", lat, lon, exc)
return {"aqi_index": 2, "aqi_label": "Unknown", "aqi_risk": 25, "pm25": 0, "pm10":
0}def get_route_aqi_risk(origin_lat, origin_lon, dest_lat, dest_lon) -> dict:
"""Return the higher AQI risk between origin and destination."""    origin_aqi =
get_aqi(origin_lat, origin_lon)    dest_aqi    = get_aqi(dest_lat, dest_lon)    if
dest_aqi["aqi_risk"] >= origin_aqi["aqi_risk"]:        worst = dest_aqi
worst["location"] = "destination"    else:        worst = origin_aqi
worst["location"] = "origin"    return worst
```

traffic_service.py — Traffic Risk Computation

```
# traffic_service.py - Compute traffic risk score for city pairsimport loggingfrom
datetime import datetimefrom zoneinfo import ZoneInfologger =
logging.getLogger(__name__)IST = ZoneInfo("Asia/Kolkata")# Base congestion indices per
route (0-100 scale)BASE_CONGESTION = {    ("Mumbai", "Pune"):            65,
("Delhi", "Agra"):             55,    ("Bangalore", "Chennai"):      50,
("Hyderabad", "Vijayawada"):   45,    ("Kolkata", "Bhubaneswar"):    40,
("Ahmedabad", "Surat"):        50,    ("Jaipur", "Jodhpur"):         35,    ("Delhi",
"Chandigarh"):       55,}# Time-of-day multipliers (hour: multiplier)HOUR_MULTIPLIERS
= {    range(0, 5):   0.3,   # Late night    range(5, 8):   0.6,   # Early morning
range(8, 10):  1.4,   # Morning peak    range(10, 17): 0.9,   # Off-peak day
range(17, 21): 1.5,   # Evening peak    range(21, 24): 0.5,   # Night}def
get_hour_multiplier(hour: int) -> float:    for r, mult in HOUR_MULTIPLIERS.items():
if hour in r:            return mult    return 1.0def get_traffic_risk(origin: str,
destination: str) -> dict:    """Compute a traffic risk score (0-100) for a given
route."""    now = datetime.now(IST)    hour = now.hour    weekday = now.weekday()  #
0=Monday, 6=Sunday    base = BASE_CONGESTION.get(        (origin, destination),
BASE_CONGESTION.get((destination, origin), 50)    )    multiplier =
get_hour_multiplier(hour)    if weekday >= 5:      # Weekend        multiplier *= 0.7
if weekday == 4:      # Friday evening        multiplier *= 1.1    raw_score =
min(100, base * multiplier)    if raw_score < 30:        label = "Light"    elif
raw_score < 55:        label = "Moderate"    elif raw_score < 75:        label =
"Heavy"    else:        label = "Severe"    logger.info("Traffic risk for %s->%s: %.1f
(%s) at hour %d",                origin, destination, raw_score, label, hour)
return {        "traffic_risk":  round(raw_score, 1),        "traffic_label": label,
"hour":          hour,        "weekday":       weekday,    }
```

risk_engine.py — Travel Risk Score and Advisory Generation

```
# risk_engine.py - Compute Travel Risk Score and generate advisory textimport
loggingfrom config import WEIGHT_WEATHER, WEIGHT_AQI, WEIGHT_TRAFFIC, WEIGHT_ALERTfrom
config import RISK_LOW, RISK_MODERATE, RISK_HIGHlogger = logging.getLogger(__name__)#
─── Weather Sub-Score (0-100) ──────────────────────────────────────────def
compute_weather_risk(weather: dict, alerts: list) -> float:    score = 0.0    temp =
weather.get("temp", 25)    rain = weather.get("rain_1h", 0)    wind =
weather.get("wind_speed", 0)     # m/s    vis  = weather.get("visibility", 10000)  #
metres    # Temperature component (0-40)    if temp > 45 or temp < 5:        score +=
40    elif temp > 42 or temp < 8:        score += 30    elif temp > 38 or temp < 12:
score += 20    elif temp > 35 or temp < 15:        score += 10    # Precipitation
component (0-35)    if rain > 30:        score += 35    elif rain > 15:        score
+= 25    elif rain > 7:        score += 15    elif rain > 2:        score += 8    #
Wind component (0-15)    wind_kmh = wind * 3.6    if wind_kmh > 90:        score += 15
elif wind_kmh > 60:        score += 10    elif wind_kmh > 40:        score += 5    #
Visibility component (0-10)    if vis < 500:        score += 10    elif vis < 1500:
score += 7    elif vis < 3000:        score += 4    return min(100, score)# ─── Alert
Severity Sub-Score (0-100) ────────────────────────────────────────def
compute_alert_risk(alerts: list) -> float:    if not alerts:        return 0.0
```

```
severity_map = {"minor": 25, "moderate": 50, "severe": 75, "extreme": 100}    return
max(severity_map.get(a.get("severity", "minor"), 0) for a in alerts)# ── Composite
Risk Score ──────────────────────────────────────────────────────def
compute_travel_risk_score(weather_risk: float, aqi_risk: float,
traffic_risk: float, alert_risk: float) -> float:    score = (        weather_risk  *
WEIGHT_WEATHER +        aqi_risk      * WEIGHT_AQI    +        traffic_risk  *
WEIGHT_TRAFFIC +        alert_risk    * WEIGHT_ALERT    )    return round(min(100,
max(0, score)), 1)def classify_risk(score: float) -> str:    if score <= RISK_LOW:
return "Low"    elif score <= RISK_MODERATE:        return "Moderate"    elif score <=
RISK_HIGH:        return "High"    else:        return "Critical"# ── Advisory Text
Generation ──────────────────────────────────────────────────────def
generate_advisory(origin: str, destination: str, score: float,
risk_level: str, weather: dict, aqi: dict,                              traffic: dict,
alerts: list) -> str:    lines = [        f"TRAVEL ADVISORY: {origin} ->
{destination}",        f"Travel Risk Score: {score:.0f} / 100  |  Risk Level:
{risk_level}",        "",    ]    if risk_level == "Low":        lines.append("Status:
CONDITIONS FAVORABLE. Safe to travel with standard precautions.")    elif risk_level
== "Moderate":        lines.append("Status: EXERCISE CAUTION. Monitor conditions
before and during travel.")    elif risk_level == "High":        lines.append("Status:
HIGH RISK. Delay non-essential travel. Plan for contingencies.")    else:
lines.append("Status: CRITICAL RISK. Avoid travel if possible. Follow official
advisories.")    lines.append("--- CURRENT CONDITIONS ---")    lines.append(f"Weather:
{weather.get('weather_desc','N/A').title()}")    lines.append(f"Temperature:
{weather.get('temp','N/A'):.1f}°C (feels like
{weather.get('feels_like','N/A'):.1f}°C)")    lines.append(f"Wind Speed:
{weather.get('wind_speed',0)*3.6:.1f} km/h")    lines.append(f"Rainfall:
{weather.get('rain_1h',0):.1f} mm/hr")    lines.append(f"Visibility:
{weather.get('visibility',10000)/1000:.1f} km")    lines.append(f"Air Quality:
{aqi.get('aqi_label','N/A')} (AQI Index: {aqi.get('aqi_index','N/A')})")
lines.append(f"PM2.5:       {aqi.get('pm25',0):.1f} µg/m³")    lines.append(f"Traffic:
{traffic.get('traffic_label','N/A')} (Score:
{traffic.get('traffic_risk',0):.0f}/100)")    if alerts:        lines.append("---
ACTIVE ALERTS ---")        for a in alerts:            lines.append(f"
[{a['severity'].upper()}] {a['event']}")            if a.get("description"):
lines.append(f"  {a['description'][:200]}...")    lines.append("--- RECOMMENDATIONS --
-")    recs = build_recommendations(risk_level, weather, aqi, traffic, alerts)    for
r in recs:        lines.append(f"  • {r}")    return "".join(lines)def
build_recommendations(risk_level, weather, aqi, traffic, alerts):    recs = []    rain
= weather.get("rain_1h", 0)    temp = weather.get("temp", 25)    wind_kmh =
weather.get("wind_speed", 0) * 3.6    vis = weather.get("visibility", 10000)
aqi_idx = aqi.get("aqi_index", 1)    traf = traffic.get("traffic_risk", 0)    if
risk_level in ("High", "Critical"):        recs.append("Consider postponing non-
essential travel until conditions improve.")    if rain > 15:
recs.append("Heavy rainfall expected: reduce speed, increase following distance.")
recs.append("Carry rain gear and waterproof bags for luggage.")    if rain > 30:
recs.append("Flash flood risk: avoid low-lying roads and river crossings.")    if temp
> 40:        recs.append("Extreme heat: carry at least 3 litres of water; check
vehicle coolant.")    if temp < 10:        recs.append("Cold conditions: check brakes,
carry warm clothing and emergency blanket.")    if wind_kmh > 60:
recs.append("Strong winds: high-sided vehicles should exercise extreme caution.")
if vis < 2000:        recs.append("Poor visibility: use fog lights, reduce speed
significantly.")    if aqi_idx >= 4:        recs.append("Poor air quality: use N95
mask; keep vehicle cabin air on recirculate.")    if aqi_idx == 5:
recs.append("Very poor AQI: vulnerable individuals (elderly, children, respiratory
conditions) should avoid travel.")    if traf > 60:        recs.append("Heavy traffic
expected: depart earlier or later to avoid peak congestion.")    if alerts:
recs.append("Active weather alerts in effect: monitor IMD and local authority
updates.")    if not recs:        recs.append("Standard road safety precautions
apply.")    return recs
```

notification_service.py — DynamoDB Storage and SNS Delivery

```
# notification_service.py - Store advisories in DynamoDB and notify via SNSimport
boto3import loggingimport uuidfrom datetime import datetimefrom zoneinfo import
ZoneInfofrom config import AWS_REGION, SNS_TOPIC_ARN, DYNAMODB_TABLE, RISK_LOWlogger =
logging.getLogger(__name__)IST = ZoneInfo("Asia/Kolkata")dynamodb =
boto3.resource("dynamodb", region_name=AWS_REGION)sns_client = boto3.client("sns",
region_name=AWS_REGION)table = dynamodb.Table(DYNAMODB_TABLE)def
store_advisory(origin: str, destination: str, risk_score: float,
risk_level: str, advisory_text: str,                    weather: dict, aqi: dict,
traffic: dict) -> str:    """Persist travel advisory record to DynamoDB."""
record_id = str(uuid.uuid4())    timestamp = datetime.now(IST).isoformat()    route_id
= f"{origin.replace(' ','_')}_{destination.replace(' ','_')}"    try:
table.put_item(Item={            "record_id":     record_id,            "route_id":
route_id,            "timestamp":     timestamp,            "origin":        origin,
"destination":   destination,            "risk_score":    str(round(risk_score, 1)),
"risk_level":    risk_level,            "advisory_text": advisory_text,
"temp":          str(weather.get("temp", "")),            "rain_1h":
str(weather.get("rain_1h", 0)),            "wind_speed":
str(weather.get("wind_speed", 0)),               "aqi_index":
str(aqi.get("aqi_index", "")),            "aqi_label":     aqi.get("aqi_label", ""),
"traffic_score": str(traffic.get("traffic_risk", "")),        })
logger.info("Advisory stored: %s  route=%s  level=%s",                    record_id,
route_id, risk_level)        return record_id    except Exception as exc:
logger.error("DynamoDB put_item failed: %s", exc)        return ""def
send_notification(origin: str, destination: str,                        risk_level:
str, advisory_text: str) -> None:    """Send SNS notification for Moderate+ risk
advisories."""    if risk_level == "Low":        logger.info("Skipping SNS for Low-
risk advisory: %s->%s", origin, destination)        return    subject = (
f"[{risk_level.upper()} TRAVEL RISK] {origin} to {destination} Advisory"    )    try:
response = sns_client.publish(            TopicArn=SNS_TOPIC_ARN,
Subject=subject,            Message=advisory_text,        )        logger.info("SNS
published for %s->%s: MessageId=%s",                    origin, destination,
response["MessageId"])    except Exception as exc:        logger.error("SNS publish
failed for %s->%s: %s", origin, destination, exc)
```

lambda_function.py — Main Orchestration Entry Point

```
# lambda_function.py - AWS Lambda entry point for Travel Advisory Systemimport
jsonimport loggingfrom datetime import datetimefrom zoneinfo import ZoneInfofrom
config import CITIES_CONFIG_FILEfrom weather_service import get_weather,
get_weather_alertsfrom aqi_service import get_route_aqi_riskfrom traffic_service
import get_traffic_riskfrom risk_engine import (compute_weather_risk,
compute_alert_risk,                            compute_travel_risk_score, classify_risk,
generate_advisory)from notification_service import store_advisory,
send_notificationlogging.basicConfig(level=logging.INFO)logger =
logging.getLogger(__name__)IST = ZoneInfo("Asia/Kolkata")def load_cities() ->
list[dict]:    with open(CITIES_CONFIG_FILE, "r") as f:        return
json.load(f)["routes"]def process_route(route: dict) -> dict | None:    """Process a
single city-pair route and return advisory result."""    origin      = route["origin"]
destination = route["destination"]    logger.info("Processing route: %s -> %s",
origin, destination)    # 1. Fetch weather data for both cities    origin_weather =
get_weather(origin)    dest_weather   = get_weather(destination)    if not
origin_weather or not dest_weather:        logger.warning("Weather data unavailable
for %s->%s; skipping.", origin, destination)        return None    # Use destination
weather as primary for advisory (traveler endpoint)    weather = dest_weather
weather["rain_1h"] = max(        origin_weather.get("rain_1h", 0),
dest_weather.get("rain_1h", 0)    )    # 2. Fetch weather alerts for both endpoints
origin_alerts = get_weather_alerts(origin_weather["lat"], origin_weather["lon"])
dest_alerts   = get_weather_alerts(dest_weather["lat"], dest_weather["lon"])
all_alerts    = origin_alerts + dest_alerts    # 3. Fetch AQI (worst of
origin/destination)    aqi = get_route_aqi_risk(        origin_weather["lat"],
origin_weather["lon"],        dest_weather["lat"], dest_weather["lon"]    )    # 4.
Compute traffic risk    traffic = get_traffic_risk(origin, destination)    # 5.
```

```
Compute sub-scores and composite score    weather_risk  =
compute_weather_risk(weather, all_alerts)    alert_risk    =
compute_alert_risk(all_alerts)    aqi_risk      = aqi["aqi_risk"]    traffic_risk  =
traffic["traffic_risk"]    risk_score    = compute_travel_risk_score(weather_risk,
aqi_risk, traffic_risk, alert_risk)    risk_level    = classify_risk(risk_score)    #
6. Generate advisory text    advisory_text = generate_advisory(origin, destination,
risk_score, risk_level,                                    weather, aqi, traffic,
all_alerts)    # 7. Store in DynamoDB    record_id = store_advisory(origin,
destination, risk_score, risk_level,                                advisory_text,
weather, aqi, traffic)    # 8. Notify via SNS if Moderate or above
send_notification(origin, destination, risk_level, advisory_text)    return {
"route":       f"{origin} -> {destination}",        "risk_score":  risk_score,
"risk_level":  risk_level,        "record_id":   record_id,    }def
lambda_handler(event, context):    """AWS Lambda entry point triggered by EventBridge
schedule."""    start_time = datetime.now(IST)    logger.info("Travel Advisory System
invoked at %s", start_time.isoformat())    try:        routes  = load_cities()
results = []        errors  = 0        for route in routes:            try:
result = process_route(route)                if result:
results.append(result)            except Exception as exc:
logger.error("Unhandled error processing route %s->%s: %s",
route.get("origin"), route.get("destination"), exc)                errors += 1
end_time = datetime.now(IST)        duration = (end_time - start_time).total_seconds()
logger.info("Advisory cycle complete. Routes processed: %d, Errors: %d, Duration:
%.1fs",                    len(results), errors, duration)        return {
"statusCode": 200,            "body": json.dumps({                "message":
"Travel advisory cycle completed.",                "processed":  len(results),
"errors":     errors,               "duration_s": round(duration, 1),
"results":    results,            })        }    except Exception as exc:
logger.critical("Fatal error in lambda_handler: %s", exc)        return {"statusCode":
500, "body": json.dumps({"error": str(exc)})}
```

cities.json — City Pair Configuration

```
{  "routes": [    {"origin": "Mumbai",    "destination": "Pune"},    {"origin":
"Delhi",     "destination": "Agra"},    {"origin": "Bangalore", "destination":
"Chennai"},    {"origin": "Hyderabad", "destination": "Vijayawada"},    {"origin":
"Kolkata",   "destination": "Bhubaneswar"},    {"origin": "Ahmedabad", "destination":
"Surat"},    {"origin": "Jaipur",    "destination": "Jodhpur"},    {"origin": "Delhi",
"destination": "Chandigarh"}  ]}
```

requirements.txt

```
boto3>=1.34.0requests>=2.31.0# All other dependencies (logging, json, datetime,
zoneinfo, math, uuid)# are part of Python 3.11 standard library and do not require
installation.
```

EventBridge Cron Expression

The following cron expression triggers the Lambda function every 6 hours to generate advisories for all four daily travel windows:

```
cron(0 */6 * * ? *)# Triggers at: 00:00 UTC (05:30 IST), 06:00 UTC (11:30 IST),#
12:00 UTC (17:30 IST), 18:00 UTC (23:30 IST)
```

IAM Policy for Lambda Execution Role

```
{  "Version": "2012-10-17",  "Statement": [    {      "Sid": "CloudWatchLogs",
"Effect": "Allow",      "Action":
["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents"],      "Resource":
"arn:aws:logs:*:*:*"    },    {      "Sid": "DynamoDBAdvisories",      "Effect":
"Allow",      "Action": ["dynamodb:PutItem","dynamodb:Query","dynamodb:GetItem"],
"Resource": "arn:aws:dynamodb:ap-south-1:ACCOUNT_ID:table/TravelAdvisories"    },    {
"Sid": "SNSPublish",      "Effect": "Allow",      "Action": "sns:Publish",
"Resource": "arn:aws:sns:ap-south-1:ACCOUNT_ID:TravelAdvisoryTopic"    }  ]}
```

DynamoDB Table Schema

```
Table Name:    TravelAdvisoriesPartition Key: route_id   (String)  e.g.
"Mumbai_Pune"Sort Key:      timestamp  (String)  ISO-8601 with timezoneBilling Mode:
PAY_PER_REQUEST (On-Demand)TTL Attribute: expiry_epoch  (Number, Unix timestamp, 30-
day TTL)
```